

Ray Tracer

Isaac Zenteno

June 4, 2026

This project was created progressively over several iterations. From a very basic render to a more complete final scene with imported OBJ models, lighting, shadows, reflections, refractions, anti-aliasing, and BVH optimization, each version added a new function to the ray tracer.

1 Ray Tracer v0.1

In the first version, I created the basic structure of the ray tracer. The main goal was to generate an image pixel by pixel. For each pixel, the camera sends a ray into the scene. If the ray hits an object, that pixel gets the color of the object. If the ray does not hit anything, the pixel gets the background color.

This version helped me understand the basic idea of ray tracing. I created important classes like `Camera`, `Ray`, `Vector3D`, `Scene`, `Object3D`, and `Intersection`. The camera was responsible for generating rays, and each ray had an origin and a direction.



Figure 1: Ray Tracer v0.1 basic render.

2 Ray Tracer v0.2

In this version, I added clipping and triangles. The clipping part used the near and far values from the camera. This helped the ray tracer ignore intersections that were too close or too far away.

I also added the `Triangle` class. This was important because triangles are one of the most basic shapes in 3D graphics, and most OBJ models are made out of triangles. At this point, the render still looked simple, but the ray tracer was already checking the closest intersection for each ray.

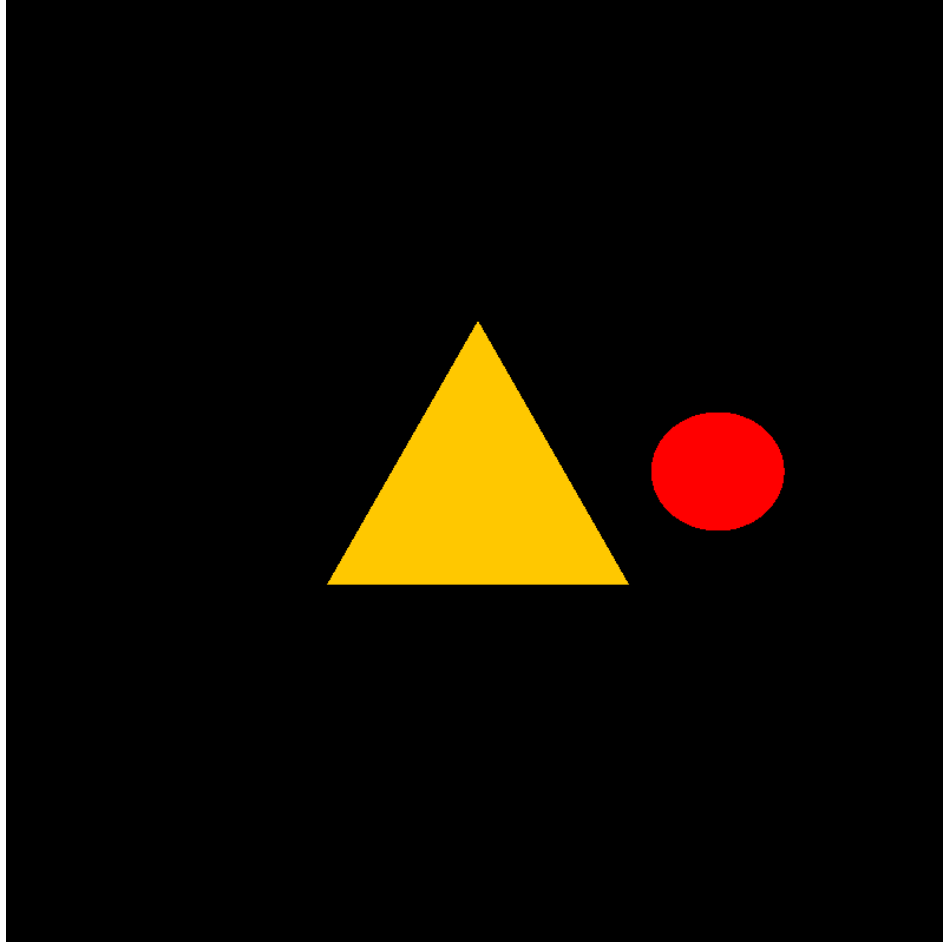


Figure 2: Ray Tracer v0.2 with clipping and triangles.

3 Ray Tracer v0.3

In v0.3, I added the OBJ loader. This allowed me to load external 3D models instead of manually creating every triangle in Java. The OBJ reader checks the vertices and faces from the file, and then converts the faces into triangles that the ray tracer can render.

This version was important because it made the project feel more like a real renderer. Instead of only rendering simple shapes, I could now render imported models.

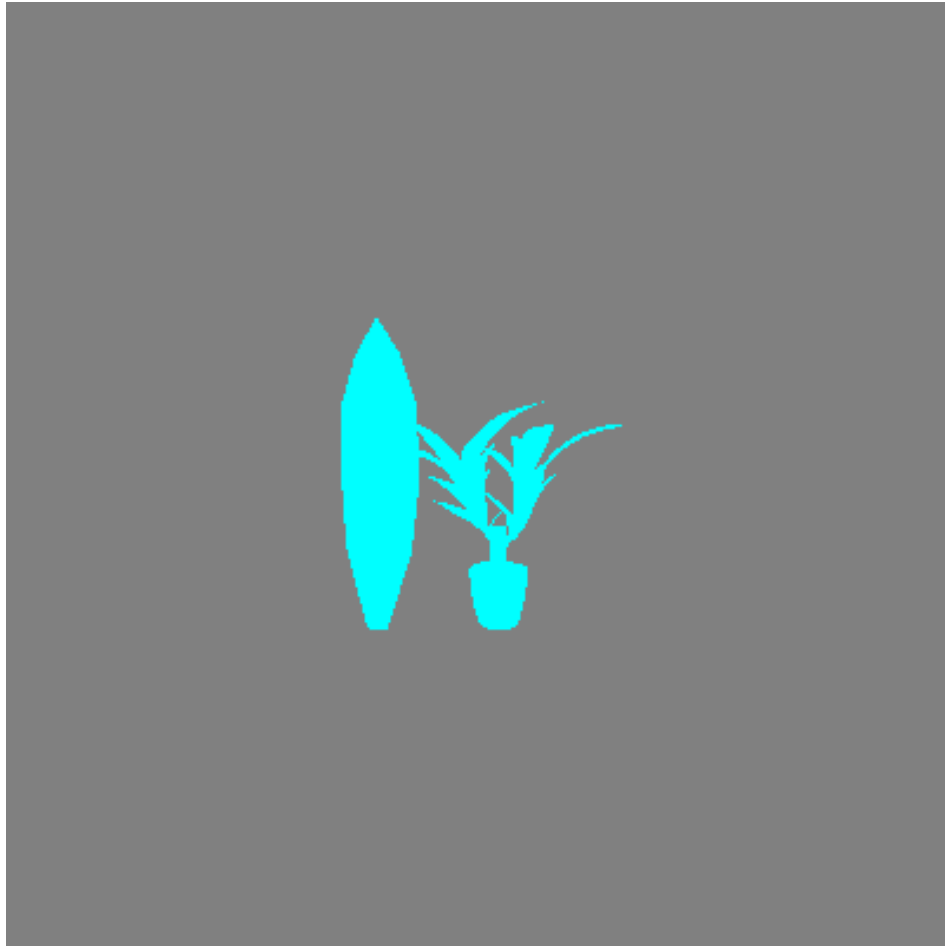


Figure 3: Ray Tracer v0.3 loading an OBJ model.

4 Ray Tracer v0.4

In v0.4, I added flat shading. Before this, the objects only had a flat base color. With flat shading, each triangle calculates its own normal, and the brightness depends on the angle between the normal and the light direction.

This made the objects look more three-dimensional because some faces were brighter and others were darker. However, since each triangle used only one normal, the render still looked a little faceted.

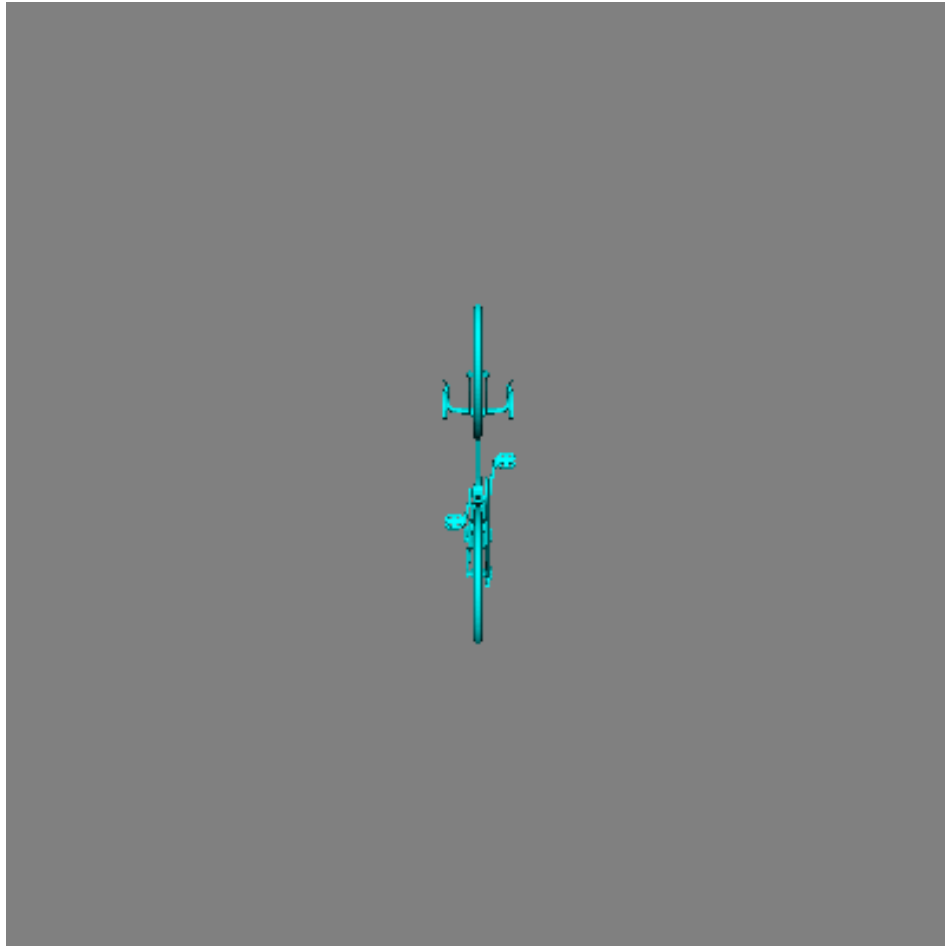


Figure 4: Ray Tracer v0.4 with flat shading.

5 Ray Tracer v0.5

In v0.5, I added Phong shading, or smooth shading. Instead of using only one normal per triangle, the ray tracer interpolates normals across the triangle. This makes the lighting look smoother and less blocky.

This helped a lot with OBJ models because they started to look more natural and less like a bunch of separate flat polygons.

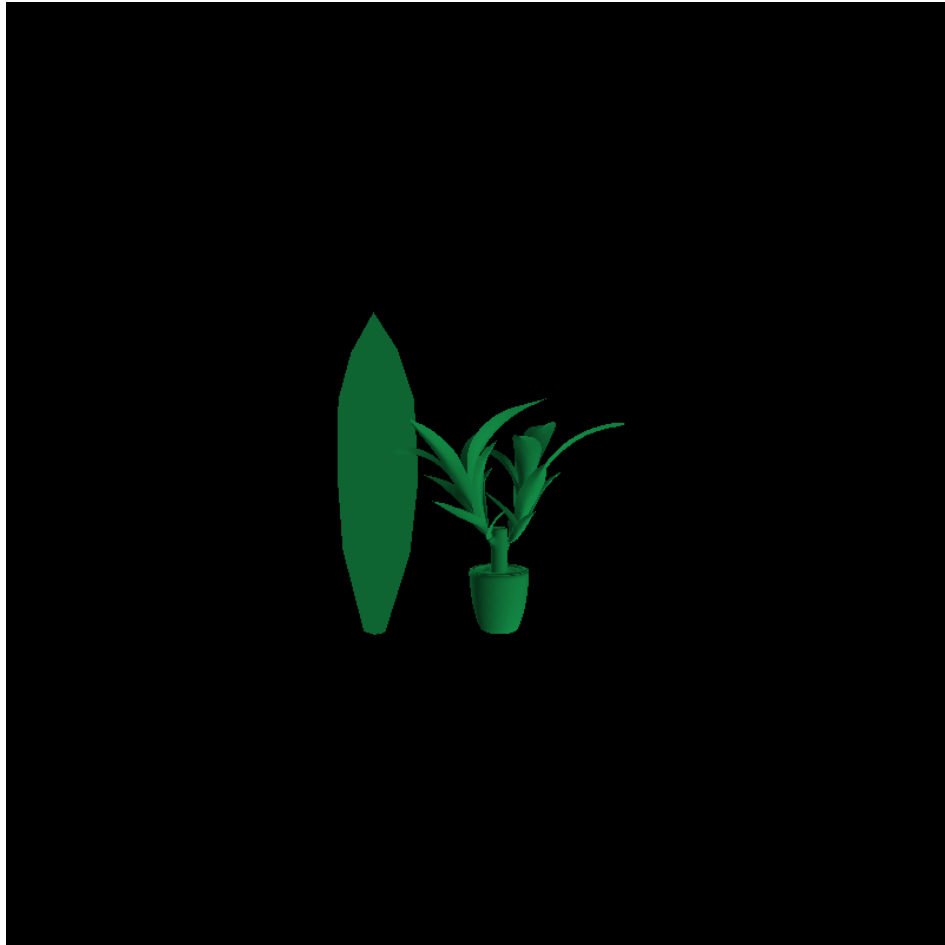


Figure 5: Ray Tracer v0.5 with Phong shading.

6 Ray Tracer v0.6

In v0.6, I added point lights with color and intensity. A point light has a position in the scene, so the light direction changes depending on the point being shaded.

This made the lighting more realistic because the light no longer came from only one fixed direction. The lights could also have different colors and intensities.

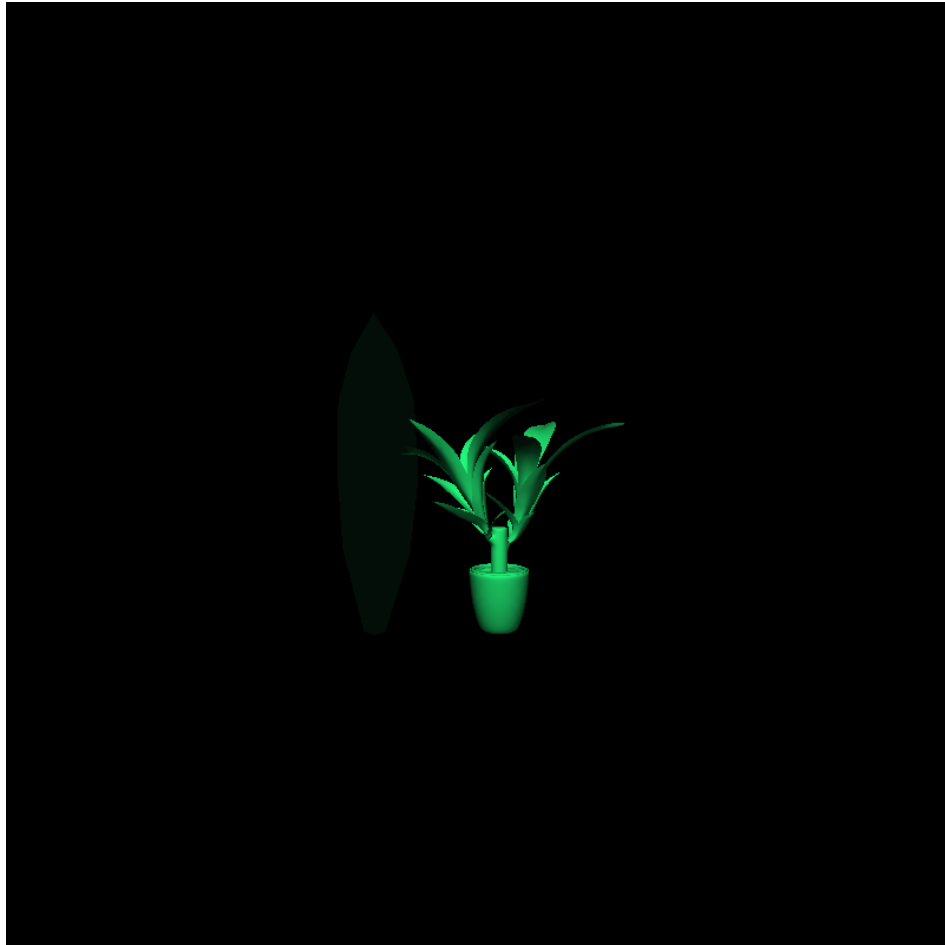


Figure 6: Ray Tracer v0.6 with point lights.

7 Ray Tracer v0.7

In v0.7, I added light falloff and shadows. Light falloff means that the farther an object is from a point light, the weaker the light becomes. This made the lighting feel more natural.

For shadows, the ray tracer sends a shadow ray from the intersection point toward the light. If that ray hits another object before reaching the light, then the point is in shadow. This helped the objects feel more grounded in the scene.

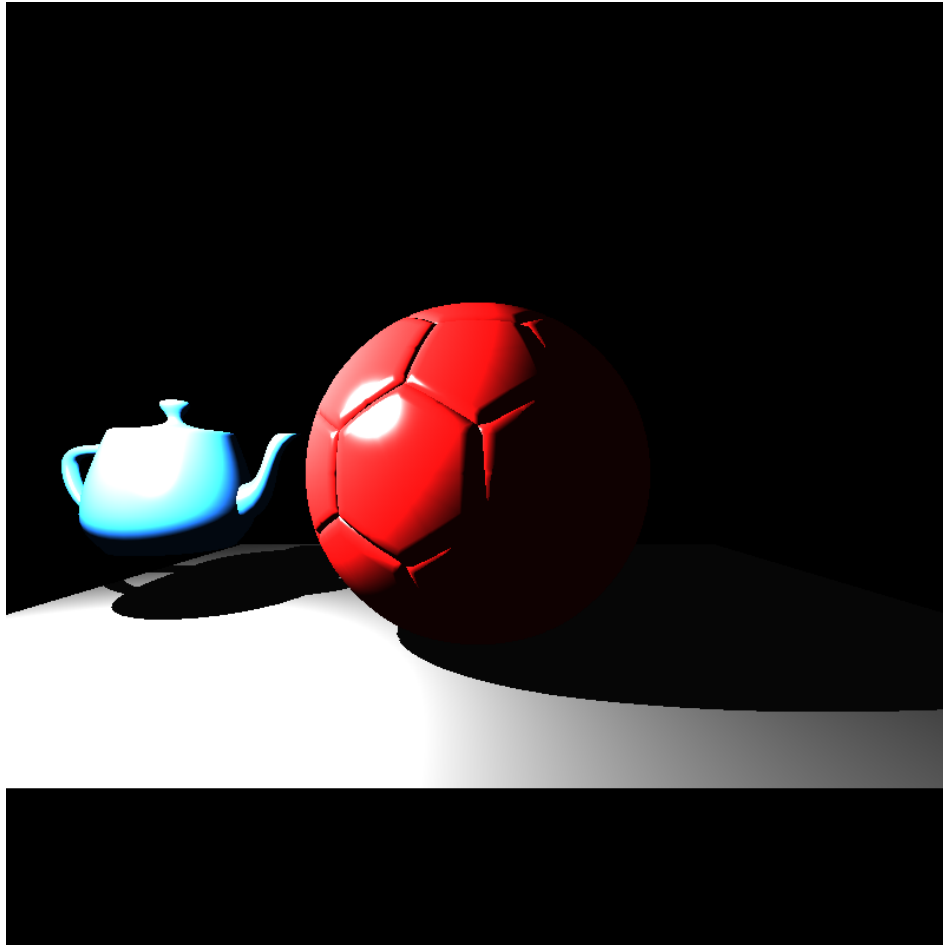


Figure 7: Ray Tracer v0.7 with light falloff and shadows.

8 Explanation of the Final Images

8.1 Final Render 1 - Triathlon

This render was part of my final scene development, and in this one I wanted the composition to feel cleaner and more organized. Instead of placing many objects together in a more crowded way, I separated the objects and displayed them on individual platforms, almost like a small exhibition room.

The scene includes three main objects that connect directly to the triathlon theme. On the left side, I placed the running shoes, which represent the running part of the triathlon. In the center, I placed the swimming goggles, which represent the swimming part. On the right side, I placed the bicycle and helmet, which represent the cycling part. By arranging the objects this way, the scene makes the triathlon idea much clearer, because each discipline is represented in a simple and direct way.

One thing I wanted to show in this render was the use of reflections. The floor is reflective, so the platforms and the objects are mirrored below them. This helps the scene look more polished and also shows one of the ray tracer features more clearly. The reflections are not as dramatic as in some other scenes, but they still help give the image more depth.

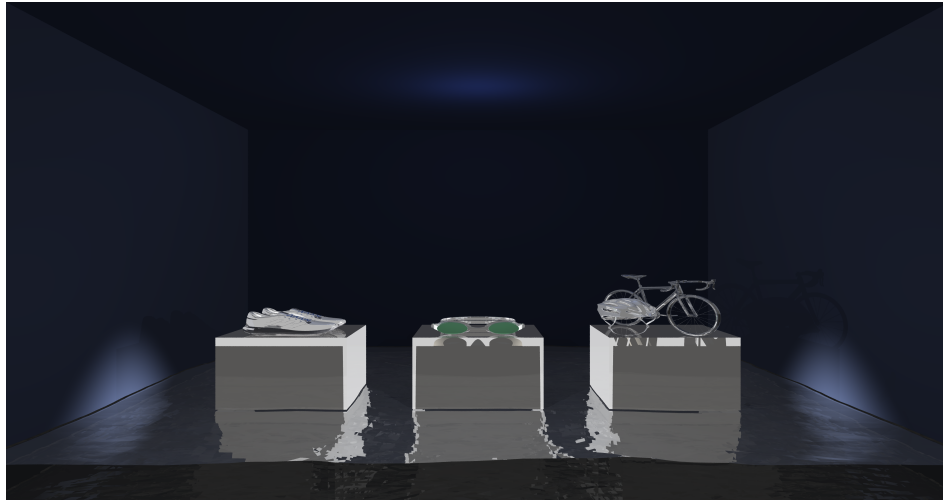


Figure 8: Render 1.

8.2 Final Render 2 — Trophy Room Scene

This render was one of my final test scenes where I wanted to show a more cinematic environment. The scene is inside a dark trophy room with several trophies placed on small bases, and a larger emblem or medal in the back wall. I used a very dark environment so the lights and reflections would stand out more.

One of the main things I wanted to show in this image was reflection. The floor is reflective, so the trophies and the room are mirrored on the surface. This helped the scene look less empty and made it feel more realistic. The reflection also shows that the ray tracer is not only coloring the objects directly, but also sending secondary rays to calculate what appears on reflective surfaces.

The lighting is also important in this render. There is a light source near the top that creates a focused highlight and gives the scene a dramatic mood. Since the room is mostly dark, the objects are only partially visible, which makes the image feel more like a display room or a museum case.

This render also shows the use of imported OBJ models. Instead of only using simple spheres or triangles, the scene contains several detailed models like trophies and the large emblem. This was important because it tested how the ray tracer handled more complex geometry.

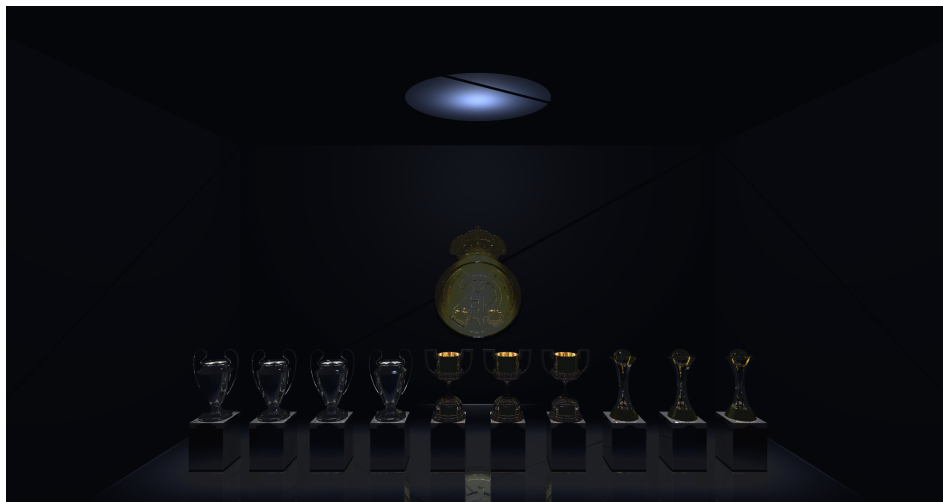


Figure 9: Render 2

8.3 Final Render 3 - Spiderman

This render was another final scene where I tested the ray tracer with a more complex environment. The scene shows a character in front of a city at night. The background has buildings, streets, lights, and a reflective floor or surface.

The main object in this image is the character in the center. I placed it in front of the city so it would stand out clearly. The pose also makes the image feel more dynamic compared to a normal still object. This was useful because it tested how the ray tracer handled a detailed model with many surfaces and colors.

The city background helped make the scene feel bigger and more complete. Instead of having the object alone in an empty space, the buildings create depth and context. The night setting also works well with the lighting because the bright areas of the street and buildings contrast with the darker parts of the image.



Figure 10: Render 3.

9 Greatest Challenges

One of the biggest challenges was understanding how the full ray tracing process works. At first, it looked like it was only about shooting a ray and coloring a pixel, but later I realized that each ray has to check intersections, calculate normals, apply lighting, check shadows, and sometimes create more rays for reflection and refraction.

Another challenge was loading OBJ models. Some models were too big, too small, or not centered correctly. Because of that, I had to improve the OBJ loader so each object could have its own position and scale.

Shadows were also difficult because I had to avoid the shadow ray hitting the same object it started from. I solved this using a small epsilon value.

Performance was another big challenge. Once I started using several OBJ models, the render became slower because there were many triangles. Adding BVH helped improve the performance.

10 Self-Evaluation

I think my project meets the main requirements. I started with a simple ray tracer and improved it version by version. I implemented camera rays, clipping, triangles, OBJ loading, flat shading, Phong shading, point lights, shadows, falloff, Blinn-Phong, reflections, refractions, anti-aliasing, and BVH.

I also applied the feedback I received. I moved the ambient light and background color into the scene, changed the OBJ loader so each object can have its own position and scale, and improved the light structure using classes like PointLight, etc.

Overall, it was a really hard project, but I learned a lot from it. I understood better how rendering works, how light interacts with objects, and how important it is to organize the code correctly. At the end, I was able to create a ray tracer that not only renders objects, but also shows lighting, shadows, reflections, refractions, anti-aliasing, and a final scene with a clear idea behind it.